

UNIVERSIDADE PRESBITERIANA MACKENZIE
FACULDADE DE COMPUTAÇÃO E INFORMÁTICA

THAINÁ SILVA LEITE (10730503)

PROJETO APLICADO III:
Sistema de Recomendação de Artigos Científicos sobre Sustentabilidade e Meio
Ambiente para Estudantes de Graduação

SÃO PAULO

2026

SUMÁRIO

1 INTRODUÇÃO	1
1.1 Contexto	1
1.2 Motivação	2
1.3 Justificativa	2
1.4 Objetivos.....	3
1.5 <i>Dataset</i> Selecionado	4
2 REFERENCIAL TEÓRICO	6
2.1 Sistemas de Recomendação.....	6
2.2 Sistemas de Recomendação Baseados em Conteúdo	6
2.3 Processamento de Linguagem Natural (PLN)	7
2.4 Vetorização Textual e TF-IDF	7
2.5 Similaridade de Cosseno	8
2.6 Sistemas de Recomendação no Contexto Educacional	8
3 METODOLOGIA.....	10
3.1 Coleta de Dados.....	11
3.2 Pré-processamento dos Dados	12
3.3 Filtragem dos Dados	13
3.4 Representação dos Dados	14
3.5 Modelo de Recomendação.....	14
3.6 Treinamento do Modelo	15
3.7 Ajustes e Refinamentos do <i>Pipeline</i>	15
3.8 Avaliação do Modelo	17
4 RESULTADOS	18
4.1 Caracterização Temporal do <i>Corpus</i>	18

4.2 Análise Lexical do <i>Corpus</i>	19
4.3 Análise da Matriz de Similaridade	22
4.4 Pares de Maior Similaridade e Coerência das Recomendações	25
4.5 Estrutura Semântica do <i>Corpus</i> (PCA).....	26
4.6 Avaliação Qualitativa das Recomendações	27
5 DISCUSSÃO	28
5.1 Eficácia e Limites do Modelo de Espaço Vetorial	28
5.2 Ponderação de Campos e Estratégias de Pré-processamento	29
5.3 Limitações do Modelo e Necessidade de Representações Semânticas	30
5.4 Avaliação Quantitativa e Trabalhos Futuros	29
6 CONCLUSÃO	32
REFERÊNCIAS	33
ANEXO A – Código-fonte do Sistema de Recomendação.....	37
ANEXO B – Código-fonte do Script de Visualização.....	49

1 INTRODUÇÃO

1.1 Contexto

O crescimento acelerado da produção científica nas últimas décadas resultou em um volume expressivo de artigos publicados anualmente em diversas áreas do conhecimento. Estudos bibliométricos indicam um aumento contínuo no número de publicações científicas ao longo dos anos, impulsionado pela expansão das bases digitais e da colaboração internacional (Bornmann & Mutz, 2015).

No campo da sustentabilidade e do meio ambiente, esse crescimento é ainda mais significativo devido à urgência global relacionada às mudanças climáticas e ao desenvolvimento sustentável, amplamente discutidos em relatórios internacionais como os do Painel Intergovernamental sobre Mudanças Climáticas (IPCC, 2023).

Entretanto, a abundância de publicações científicas gera um desafio no ambiente educacional: a dificuldade de estudantes de graduação em identificar artigos relevantes, atualizados e alinhados ao seu nível de conhecimento e área de interesse. Esse cenário caracteriza um problema conhecido como sobrecarga informacional (*information overload*), no qual o excesso de informações disponíveis dificulta a tomada de decisão e pode comprometer o desempenho cognitivo e a qualidade do processo de aprendizagem (Bawden & Robinson, 2009; Eppler & Mengis, 2004).

Nesse contexto, os sistemas de recomendação apresentam-se como soluções tecnológicas capazes de organizar e filtrar informações de forma personalizada. Esses sistemas utilizam técnicas de mineração de dados e aprendizado de máquina para sugerir itens relevantes aos usuários com base em seus perfis e características dos conteúdos (Ricci *et al.*, 2015; Resnick & Varian, 1997).

No ambiente acadêmico, tais ferramentas podem ser aplicadas para apoiar estudantes na seleção de artigos científicos, contribuindo para a redução da sobrecarga informacional e promovendo maior eficiência na pesquisa bibliográfica e no processo de aprendizagem (Manouselis *et al.*, 2013).

1.2 Motivação

A motivação deste projeto está relacionada à necessidade de facilitar o acesso de estudantes de graduação a conteúdos científicos confiáveis e alinhados aos seus interesses na área de sustentabilidade. Estudos apontam que muitos alunos, especialmente nos primeiros anos da graduação, apresentam dificuldades na identificação, avaliação e seleção de fontes acadêmicas adequadas, seja por limitações em competências informacionais, seja pelo grande volume de informações disponíveis (Head & Eisenberg, 2010; Association of College and Research Libraries, 2016).

Nesse contexto, a implementação de um sistema de recomendação específico para artigos científicos ambientais pode contribuir para melhorar a experiência acadêmica dos estudantes, promovendo maior autonomia na busca por conhecimento e apoiando o desenvolvimento de competências relacionadas à pesquisa científica (Drachsler *et al.*, 2015).

1.3 Justificativa

A crescente complexidade dos desafios ambientais contemporâneos exige a formação de profissionais capazes de compreender e analisar criticamente evidências científicas relacionadas à sustentabilidade. Nesse contexto, o acesso qualificado à produção científica torna-se elemento central para a consolidação da aprendizagem no ensino superior (UNESCO, 2021).

Entretanto, o volume expressivo de publicações científicas disponíveis dificulta a identificação de materiais relevantes, especialmente por estudantes em fase inicial de formação

acadêmica, que ainda estão desenvolvendo competências de busca, avaliação e seleção de fontes confiáveis (Head & Eisenberg, 2010; ACRL, 2016).

Sistemas de recomendação têm sido amplamente estudados como ferramentas eficazes para reduzir a sobrecarga informacional, organizando conteúdos de forma personalizada por meio de técnicas de mineração de dados e aprendizado de máquina (Ricci *et al.*, 2015). No contexto educacional, tais sistemas podem apoiar o processo de aprendizagem ao direcionar estudantes para recursos mais adequados ao seu perfil e às suas necessidades acadêmicas (Drachsler *et al.*, 2015).

Além disso, ao focar em artigos científicos relacionados à sustentabilidade, o projeto dialoga diretamente com os Objetivos de Desenvolvimento Sustentável estabelecidos pela Organização das Nações Unidas, especialmente o ODS 4 (Educação de Qualidade) e o ODS 13 (Ação Contra a Mudança Global do Clima), contribuindo para a democratização do acesso ao conhecimento científico e para a formação de estudantes mais preparados para enfrentar desafios socioambientais contemporâneos.

1.4 Objetivos

Objetivo Geral

Desenvolver e implementar um sistema de recomendação baseado em conteúdo para sugerir artigos científicos sobre sustentabilidade e meio ambiente a estudantes de graduação.

Objetivos Específicos

- Selecionar e analisar um *dataset* adequado ao contexto de sistemas de recomendação acadêmica;
- Realizar o pré-processamento dos textos (títulos, resumos e palavras-chave);
- Aplicar técnicas de vetorização textual, como TF-IDF;
- Implementar um modelo de recomendação baseado em similaridade de cosseno;

- Avaliar o desempenho do sistema quanto à relevância das recomendações;
- Analisar os resultados obtidos e propor melhorias futuras.

1.5 Dataset Selecionado

Para o desenvolvimento do sistema de recomendação proposto, foi utilizado um *dataset* construído a partir de dados obtidos diretamente da API do OpenAlex. O OpenAlex é uma plataforma aberta de indexação científica que disponibiliza metadados acadêmicos em larga escala (Priem *et al.*, 2022), reunindo informações sobre artigos científicos, autores, instituições, periódicos e conceitos temáticos. A plataforma se destaca no contexto da ciência aberta por oferecer acesso gratuito e estruturado a publicações científicas de diferentes áreas do conhecimento.

O conjunto de dados utilizado neste trabalho foi composto por artigos científicos relacionados às áreas de sustentabilidade, mudanças climáticas e meio ambiente, permitindo a construção de um sistema de recomendação baseado em conteúdo (Beel *et al.*, 2016). A coleta foi realizada por meio de consultas temáticas na API do OpenAlex, utilizando conceitos associados ao domínio ambiental, como mudanças climáticas, sustentabilidade, biodiversidade, energias renováveis, gases de efeito estufa, escassez hídrica e desmatamento.

Para a composição do *dataset*, foram selecionados artigos publicados entre os anos de 2020 e 2024, escritos em língua inglesa e contendo resumo disponível (abstract). Posteriormente, os registros passaram por etapas de pré-processamento textual e filtragem temática, visando garantir maior aderência ao escopo do projeto e melhorar a qualidade dos dados utilizados pelo sistema de recomendação.

O *dataset* inclui, entre suas principais variáveis:

- título do artigo;
- resumo (abstract);
- autores;

- ano de publicação;
- identificador OpenAlex;
- link DOI ou página da publicação;
- conceitos temáticos associados ao artigo.

A estrutura textual dos campos de título e resumo possibilita a aplicação de técnicas de Processamento de Linguagem Natural (PLN) (Jurafsky & Martin, 2023), especialmente métodos de vetorização textual e cálculo de similaridade semântica. Essas informações foram utilizadas na construção do sistema de recomendação baseado em filtragem por conteúdo (Lops *et al.*, 2011), empregando TF-IDF e similaridade de cosseno para identificar artigos semanticamente relacionados (Baeza-Yates & Ribeiro-Neto, 2011).

A definição do subconjunto final e dos critérios de filtragem foi realizada durante a etapa de implementação, considerando aspectos como relevância temática, qualidade dos metadados e aderência ao domínio da sustentabilidade e das ciências ambientais.

2 REFERENCIAL TEÓRICO

2.1 Sistemas de Recomendação

Os sistemas de recomendação são ferramentas computacionais desenvolvidas para auxiliar usuários na identificação de itens relevantes em meio a grandes volumes de informação. Esses sistemas têm como objetivo reduzir a sobrecarga informacional, fornecendo sugestões personalizadas com base em preferências, comportamentos ou características dos itens (Ricci *et al.*, 2022).

De forma geral, os sistemas de recomendação podem ser classificados em três principais categorias: filtragem colaborativa, baseada em conteúdo e híbrida. A filtragem colaborativa utiliza o histórico de interações entre usuários e itens para identificar padrões de preferência, enquanto os sistemas híbridos combinam múltiplas abordagens para melhorar o desempenho e reduzir limitações específicas (Resnick; Varian, 1997).

No contexto deste trabalho, adotou-se a abordagem baseada em conteúdo, considerando que a proposta se fundamenta na análise textual de artigos científicos e na ausência de dados históricos de interação entre usuários e itens, o que limita a aplicação de métodos baseados em filtragem colaborativa.

2.2 Sistemas de Recomendação Baseados em Conteúdo

A recomendação baseada em conteúdo (*content-based filtering*) fundamenta-se na análise das características dos itens para sugerir novos conteúdos semelhantes àqueles previamente selecionados ou de interesse do usuário. Nesse tipo de abordagem, os itens são representados por atributos, como palavras-chave, descrições ou textos completos, permitindo a comparação entre eles por meio de medidas de similaridade (Lops *et al.*, 2011).

Uma das principais vantagens dessa abordagem é a independência de dados de outros usuários, sendo especialmente útil em cenários em que não há histórico suficiente de interações,

como em sistemas acadêmicos voltados a novos usuários (Lops *et al.*, 2011; Aggarwal, 2016). Além disso, esse modelo permite maior transparência nas recomendações, uma vez que é possível explicar os resultados com base nas características dos itens (Ricci *et al.*, 2022).

No contexto da recomendação de artigos científicos, essa abordagem mostra-se particularmente adequada, pois utiliza diretamente informações textuais como títulos e resumos, que refletem o conteúdo semântico das publicações.

2.3 Processamento de Linguagem Natural (PLN)

O Processamento de Linguagem Natural (PLN) consiste em uma área da inteligência artificial dedicada ao desenvolvimento de métodos capazes de analisar, compreender e manipular a linguagem humana em formato textual (Jurafsky; Martin, 2026).

No contexto de sistemas de recomendação baseados em conteúdo, o PLN desempenha um papel fundamental ao permitir a conversão de textos não estruturados em representações numéricas passíveis de processamento por algoritmos computacionais (Jurafsky; Martin, 2024; Manning *et al.*, 2009). Esse processo envolve etapas como limpeza textual, remoção de palavras irrelevantes (*stopwords*), tokenização e normalização dos dados (Jurafsky; Martin, 2024).

A aplicação dessas técnicas em sistemas que utilizam dados textuais, como no caso de artigos científicos, possibilita a extração de informações relevantes e a comparação eficiente entre conteúdos (Lops *et al.*, 2011; Ricci *et al.*, 2022).

2.4 Vetorização Textual e TF-IDF

Para a aplicação de algoritmos de recomendação em dados textuais, é necessária a transformação dos textos em representações numéricas (Manning *et al.*, 2009). Nesse contexto, destaca-se o TF-IDF (*Term Frequency – Inverse Document Frequency*), uma das técnicas mais utilizadas para esse fim (Salton; Buckley, 1988).

O TF-IDF mede a importância de uma palavra em um documento em relação a um conjunto de documentos. Enquanto o termo TF representa a frequência da palavra no documento, o IDF reduz o peso de palavras muito comuns no *corpus*, destacando termos mais relevantes (Salton; Buckley, 1988).

Essa técnica é amplamente utilizada em sistemas de recomendação baseados em texto, pois permite representar documentos de forma vetorial, preservando informações importantes para análise de similaridade (Manning *et al.*, 2009).

2.5 Similaridade de Cosseno

A similaridade de cosseno é uma métrica amplamente utilizada para medir o grau de similaridade entre dois vetores em um espaço multidimensional. Essa medida é calculada com base no cosseno do ângulo entre os vetores, variando de 0 (ausência de similaridade) a 1 (máxima similaridade) (Manning *et al.*, 2008; Tan *et al.*, 2006).

Em sistemas de recomendação baseados em conteúdo, a similaridade de cosseno é frequentemente aplicada para comparar documentos representados por vetores TF-IDF, permitindo identificar itens com conteúdos semanticamente semelhantes (Manning *et al.*, 2008).

No presente projeto, essa métrica foi utilizada para calcular a proximidade entre artigos científicos, possibilitando a recomendação de conteúdos relevantes com base em suas características textuais.

2.6 Sistemas de Recomendação no Contexto Educacional

A aplicação de sistemas de recomendação no contexto educacional tem crescido significativamente, especialmente com a expansão de ambientes virtuais de aprendizagem e repositórios digitais. Esses sistemas podem apoiar estudantes na seleção de conteúdos

relevantes, contribuindo para a personalização do processo de aprendizagem (Drachsler *et al.*, 2015).

No caso específico de artigos científicos, os sistemas de recomendação podem auxiliar estudantes na identificação de materiais alinhados aos seus interesses e nível de conhecimento, reduzindo a sobrecarga informacional e promovendo maior eficiência na pesquisa acadêmica (Drachsler *et al.*, 2015; Manouselis *et al.*, 2013).

Além disso, ao direcionar conteúdos relacionados à sustentabilidade e meio ambiente, o presente projeto contribui para objetivos educacionais mais amplos, alinhando-se a iniciativas globais de promoção do conhecimento científico e desenvolvimento sustentável (UNESCO, 2021; ONU, 2015).

3 METODOLOGIA

Para o desenvolvimento do modelo de recomendação, utilizou-se a linguagem Python em ambiente de computação em nuvem, por meio da plataforma Google Colab. Essa ferramenta possibilita a execução de códigos, análise de dados e visualização de resultados de forma integrada, sendo amplamente adotada em projetos de aprendizado de máquina e ciência de dados (Bisong, 2019).

O desenvolvimento do sistema passou por três versões iterativas, sendo a versão final, denominada v3.0, aquela cujos resultados são apresentados neste trabalho. Todas as etapas do projeto, incluindo coleta de dados, pré-processamento, análise exploratória e implementação do modelo de recomendação, foram documentadas em notebook interativo e disponibilizadas em repositório público no GitHub:

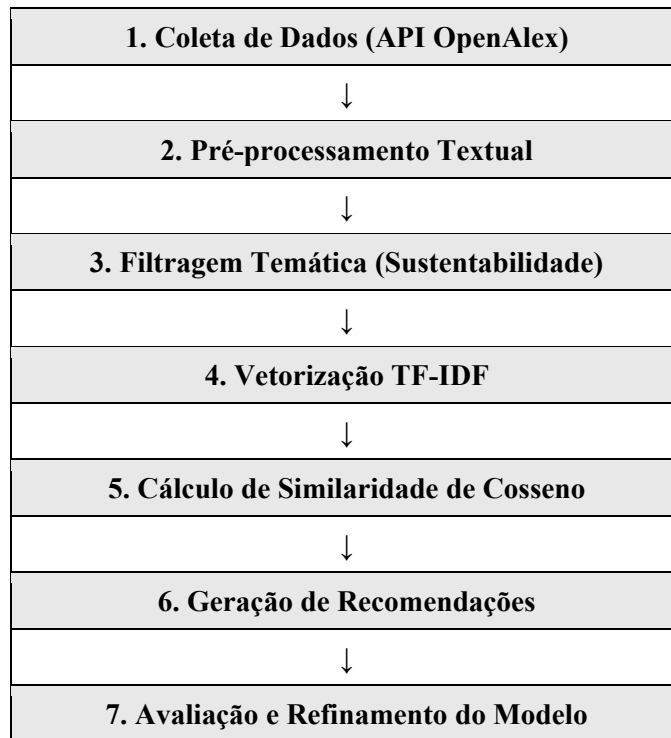
<https://github.com/slthaina/projeto-aplicado-iii>

O notebook original desenvolvido no Google Colab também pode ser acessado em:

https://colab.research.google.com/drive/17GI_I5QN9jXN1rCEwlqOmQXiH2fJ428z?usp=sharing

O *pipeline* do projeto foi estruturado de forma sequencial, conforme ilustrado no Quadro 1, abrangendo as seguintes etapas principais: coleta de dados, pré-processamento, filtragem temática, vetorização, treinamento do modelo, avaliação e refinamento.

Quadro 1. Fluxograma do *pipeline* do sistema de recomendação.



Fonte: Autoria própria.

3.1 Coleta de Dados

Para o desenvolvimento deste projeto, foi construído um *dataset* próprio a partir da API do OpenAlex, uma plataforma aberta de indexação científica voltada à disponibilização de metadados acadêmicos em larga escala. O OpenAlex reúne informações estruturadas sobre artigos científicos, autores, instituições, periódicos e conceitos temáticos, sendo amplamente utilizado em aplicações relacionadas à ciência aberta e estudos bibliométricos (Priem *et al.*, 2022).

A coleta dos dados foi realizada de forma automatizada por meio de requisições HTTP em Python utilizando a biblioteca `requests`, permitindo a comunicação direta com a API REST do OpenAlex. Entre as informações coletadas destacam-se: título do artigo, autores, resumo (abstract), ano de publicação, identificador OpenAlex, link de acesso à publicação e conceitos temáticos associados aos artigos.

As buscas foram organizadas a partir de consultas temáticas relacionadas ao domínio da sustentabilidade e das ciências ambientais. Entre os conceitos utilizados na coleta estão: “*Climate change*”, “*Sustainability*”, “*Environmental science*”, “*Renewable energy*”, “*Biodiversity*”, “*Deforestation*”, “*Greenhouse gas*”, “*Ocean acidification*”, “*Water scarcity*” e “*Wildfire*”. Esses conceitos foram selecionados com o objetivo de compor um corpus textual diversificado, abrangendo diferentes perspectivas relacionadas às questões ambientais e climáticas.

A coleta considerou artigos publicados entre os anos de 2020 e 2024, escritos em língua inglesa e contendo resumo disponível. Durante o processo, foi utilizada paginação automática da API para obtenção contínua dos registros, além de mecanismos de remoção de duplicatas com base no identificador único de cada publicação.

Após a etapa de coleta, os dados foram submetidos a procedimentos de organização e armazenamento em formato CSV, possibilitando sua utilização nas etapas posteriores de pré-processamento textual, vetorização e geração de recomendações.

3.2 Pré-processamento dos Dados

Após a etapa de coleta, foi realizado o pré-processamento dos dados com o objetivo de melhorar a qualidade das informações e prepará-las para aplicação de técnicas de Processamento de Linguagem Natural (PLN) (Jurafsky; Martin, 2024).

Inicialmente, foram removidos registros com valores nulos nas colunas essenciais, como título e resumo. Em seguida, os textos passaram por um processo de normalização, incluindo a conversão para letras minúsculas, etapa comum em tarefas de processamento textual (Manning *et al.*, 2009).

Também foi realizada limpeza textual com remoção de caracteres especiais, números e pontuações, mantendo apenas caracteres alfabéticos. Os textos foram então tokenizados,

divididos em palavras individuais, para possibilitar sua análise computacional (Jurafsky; Martin, 2024).

Para reduzir ruídos nos dados, foram removidas *stopwords* da língua inglesa — palavras muito frequentes que não agregam significado relevante, como "*the*", "*and*" e "*is*" (Manning *et al.*, 2009). Além disso, foi definida uma lista de *stopwords* customizadas, contendo termos genéricos comuns em artigos científicos, como "*model*", "*method*", "*study*", "*analysis*", "*paper*", "*result*" e "*approach*", entre outros.

Por fim, palavras muito curtas (com menos de três caracteres) também foram descartadas, visando melhorar a qualidade da representação textual (Jurafsky; Martin, 2024).

3.3 Filtragem dos Dados

Após o pré-processamento, foi aplicada uma etapa de filtragem com o objetivo de garantir maior aderência temática ao escopo do projeto.

Foi implementado um filtro baseado em palavras-chave organizado em dois grupos. O primeiro grupo, de termos fortes, inclui expressões como "*climate change*", "*global warming*", "*carbon emissions*", "*carbon footprint*", "*greenhouse gas*", "*biodiversity loss*", "*deforestation*", "*ocean acidification*", "*wildfire*" e "*water scarcity*", entre outros. O segundo grupo, de termos médios, contempla expressões como "*environment*", "*ecosystem*", "*renewable*", "*biodiversity*", "*drought*" e "*marine*", entre outros. Um artigo é considerado relevante quando contém ao menos um termo forte, ou ao menos dois termos médios em seu conteúdo. A utilização de palavras-chave como estratégia de filtragem é abordagem comum em tarefas de recuperação da informação (Manning *et al.*, 2009).

Assim, a filtragem temática possibilitou maior alinhamento do corpus ao escopo da pesquisa, reduzindo a inclusão de documentos pouco relevantes para as análises subsequentes.

3.4 Representação dos Dados

Para possibilitar o processamento computacional dos textos, foi utilizada a técnica de vetorização TF-IDF (*Term Frequency – Inverse Document Frequency*), amplamente empregada em tarefas de recuperação da informação e mineração de texto (Manning *et al.*, 2009).

Essa técnica transforma os textos em vetores numéricos, considerando a importância de cada termo em um documento em relação ao conjunto total de documentos. Dessa forma, palavras mais relevantes recebem maior peso, enquanto termos muito comuns têm sua importância reduzida (Ramos, 2003; Manning *et al.*, 2009).

Foi adotada a estratégia de atribuir maior peso ao título dos artigos, concatenando-o três vezes ao resumo durante a construção do texto final. Essa abordagem enfatiza termos-chave presentes no título, que condensam os conceitos centrais do artigo de forma mais direta (Lops *et al.*, 2010; Li, 2024). A ponderação de campos por repetição é estratégia reconhecida em recuperação da informação (Manning *et al.*, 2009).

A vetorização foi realizada com a biblioteca *scikit-learn*, com limite de até 5.000 características, uso de unigramas e bigramas, frequência mínima de documento igual a 2 e suavização logarítmica da frequência do termo (*sublinear_tf=True*) (Pedregosa *et al.*, 2011). O resultado foi uma matriz TF-IDF de 278 artigos $\times$ 4.188 características.

3.5 Modelo de Recomendação

O sistema de recomendação desenvolvido neste projeto é baseado na abordagem de filtragem por conteúdo (*content-based filtering*), amplamente utilizada em sistemas de recomendação para sugerir itens com base nas características dos próprios dados (Ricci *et al.*, 2022).

Após a vetorização TF-IDF, foi calculada a similaridade entre os documentos por meio da métrica de similaridade de cosseno, que mede o grau de proximidade entre dois vetores em um espaço multidimensional, permitindo identificar artigos semanticamente semelhantes (Manning *et al.*, 2009).

A partir da matriz de similaridade, foi implementada uma função de recomendação que, dado um artigo de entrada, retorna os artigos mais similares ordenados pelo grau de similaridade, com aplicação de score mínimo de 0,10 para evitar retorno de sugestões semanticamente distantes. Adicionalmente, foi implementada função de busca por palavra-chave nos resumos, prática comum em sistemas de recuperação da informação (Manning *et al.*, 2009).

3.6 Treinamento do Modelo

O treinamento do modelo consiste na geração da matriz TF-IDF e no cálculo da matriz de similaridade de cosseno entre os artigos (Manning *et al.*, 2009).

Como se trata de um sistema baseado em conteúdo, não há processo de treinamento supervisionado tradicional. O modelo constrói uma representação vetorial dos documentos e utiliza essa estrutura para calcular similaridades, característica típica de abordagens de filtragem por conteúdo (Ricci *et al.*, 2015). Essa abordagem foi adotada como prova de conceito, permitindo validar a viabilidade da recomendação de artigos científicos com base em características textuais.

3.7 Ajustes e Refinamentos do Pipeline

Com base nos resultados preliminares obtidos na prova de conceito, foram realizados ajustes no pipeline de treinamento visando à melhoria do desempenho do sistema.

O primeiro refinamento consistiu na expansão e revisão da lista de *stopwords* customizadas. Termos como "*paper*", "*result*", "*proposed*", "*approach*" e "*based*" foram

adicionados à lista, reduzindo o ruído causado por vocabulário genérico presente em qualquer artigo científico e melhorando a discriminação temática entre os documentos.

O segundo ajuste envolveu a revisão dos critérios de filtragem temática, com a ampliação da lista de termos fortes, incorporando expressões como "*carbon footprint*", "*greenhouse gas*", "*ocean acidification*", "*wildfire*" e "*water scarcity*". Essa modificação permitiu ampliar a cobertura de subtemas relevantes à sustentabilidade e aumentar a sensibilidade do processo de seleção dos artigos.

O terceiro refinamento foi a normalização L2 nos vetores TF-IDF antes do cálculo da similaridade de cosseno. Embora a similaridade de cosseno seja invariante à magnitude, a normalização explícita garante consistência numérica e melhora a estabilidade dos resultados em implementações via *scikit-learn* (Pedregosa *et al.*, 2011).

O quarto refinamento consistiu na revisão da estratégia de ponderação do título, passando de duplicação para triplicação do campo antes da concatenação com o resumo. Essa abordagem fundamenta-se no princípio de que documentos podem ser representados por múltiplos campos com diferentes níveis de relevância informacional (Manning *et al.*, 2009), sendo o título o campo que condensa os conceitos centrais de forma mais objetiva (Lops *et al.*, 2011; Li, 2024).

Por fim, o quinto refinamento foi a expansão das consultas de coleta de dados de cinco para onze queries, com adição de subtemas distintos ("*ocean acidification*", "*wildfire ecology*" e "*water scarcity drought*") para diversificar o *corpus* sem sobreposição, e inclusão de filtro pós-coleta para garantir que apenas artigos publicados dentro do período 2022–2026 fossem retidos.

3.8 Avaliação do Modelo

A avaliação do sistema foi realizada de forma qualitativa, considerando a relevância das recomendações geradas, abordagem comum em fases iniciais de desenvolvimento de sistemas de recomendação (Ricci *et al.*, 2015).

Foram analisados os seguintes aspectos: coerência temática entre os artigos recomendados; similaridade semântica entre os conteúdos; e aderência ao domínio de sustentabilidade. Adicionalmente, foram observadas métricas da matriz de similaridade, utilizadas para avaliar o comportamento geral do corpus e a distribuição das relações semânticas entre os artigos.

Os resultados decorrentes desses ajustes são apresentados e discutidos nas seções 4 e 5 deste trabalho.

4 RESULTADOS

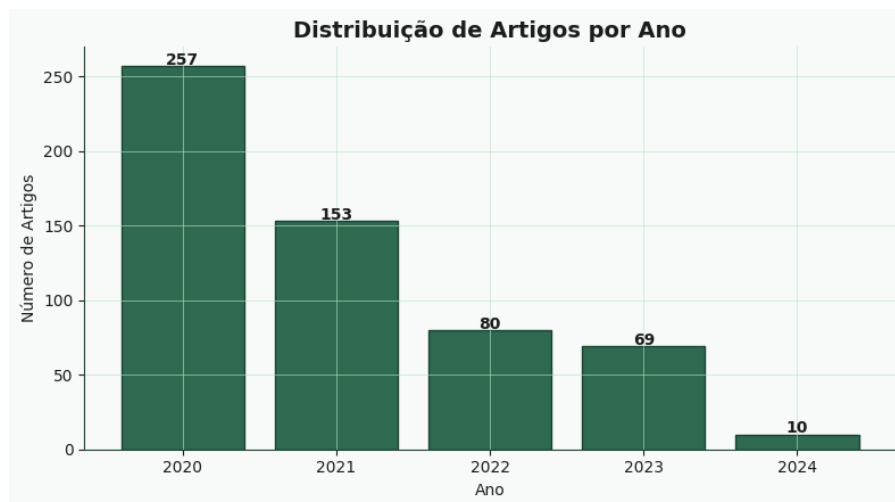
O sistema foi avaliado a partir de um corpus final composto por 569 artigos coletados via API do OpenAlex, abrangendo publicações do período de 2020 a 2024. Após a aplicação das etapas de pré-processamento e filtragem temática de sustentabilidade, o conjunto resultante apresentou vocabulário técnico aderente ao domínio ambiental, evidenciando a efetividade da estratégia de seleção baseada em palavras-chave na delimitação do escopo temático da pesquisa.

A etapa de filtragem reduziu o conjunto inicial de 1000 para 569 artigos, indicando que parte significativa dos registros recuperados inicialmente não apresentava aderência suficiente aos critérios temáticos estabelecidos. Os trabalhos selecionados concentraram-se principalmente em discussões relacionadas a mudanças climáticas, biodiversidade, emissões de carbono, recursos naturais e energias renováveis, mantendo alinhamento com os objetivos do sistema de recomendação proposto.

4.1 Caracterização Temporal do *Corpus*

A distribuição dos artigos ao longo dos anos de publicação evidencia a concentração do *corpus* no intervalo definido nos critérios de coleta, com variação no volume de publicações entre os anos. A análise temporal permite verificar se há viés de cobertura em algum subperíodo específico (Figura 1).

Figura 1. Distribuição de artigos por ano de publicação no *corpus* filtrado.

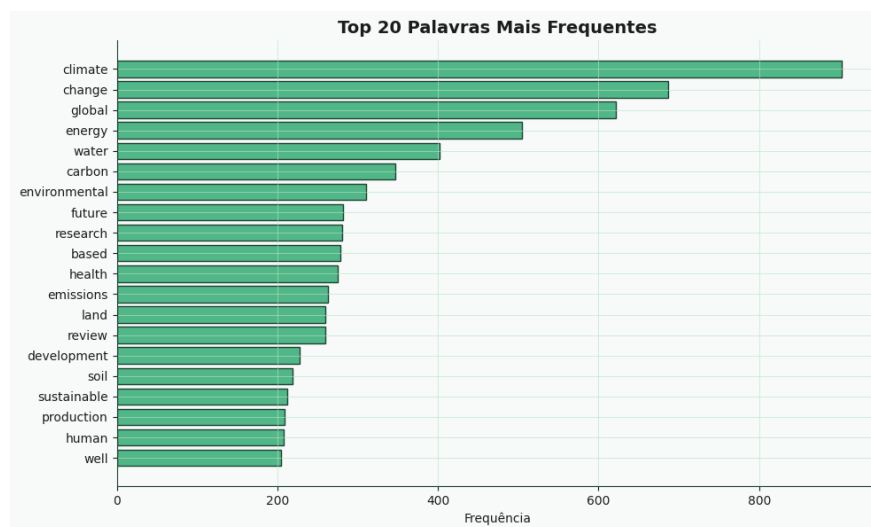


Fonte: Autoria própria.

4.2 Análise Lexical do *Corpus*

A análise de frequência dos termos presentes nos resumos revela o vocabulário dominante do *corpus* após o pré-processamento. Os vinte termos mais recorrentes concentram-se em conceitos centrais ao domínio da sustentabilidade ambiental, como *climate*, *energy*, *emissions*, *environmental* e *carbon*, o que reforça a coerência temática do conjunto resultante após as etapas de filtragem (Figura 2).

Figura 2. Vinte palavras mais frequentes nos resumos dos artigos após pré-processamento.



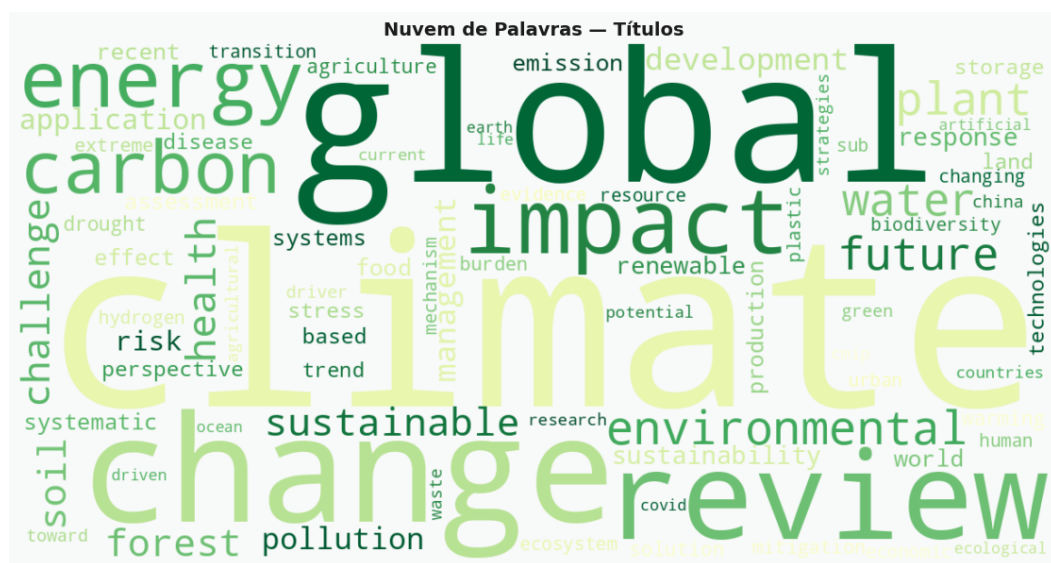
Fonte: Autoria própria.

A representação visual por nuvem de palavras complementa a análise lexical ao evidenciar, de forma proporcional, os termos de maior frequência presentes nos títulos e resumos do corpus analisado. Nas duas representações, observa-se predominância de termos como *climate*, *change*, *global*, *energy*, *environmental*, *impact* e *carbon*, indicando forte aderência temática às discussões sobre mudanças climáticas, sustentabilidade ambiental e transição energética.

Nos títulos, os termos aparecem de maneira mais concentrada e objetiva, refletindo os principais eixos temáticos utilizados na coleta dos artigos. Já nos resumos, verifica-se maior diversidade lexical, com a presença de termos relacionados a subáreas específicas, como *emission*, *ecosystem*, *land*, *health*, *species*, *resource* e *temperature*. Essa diferença sugere que, embora os artigos compartilhem um núcleo temático comum, os conteúdos abordam múltiplas dimensões ambientais, sociais e econômicas associadas às mudanças climáticas.

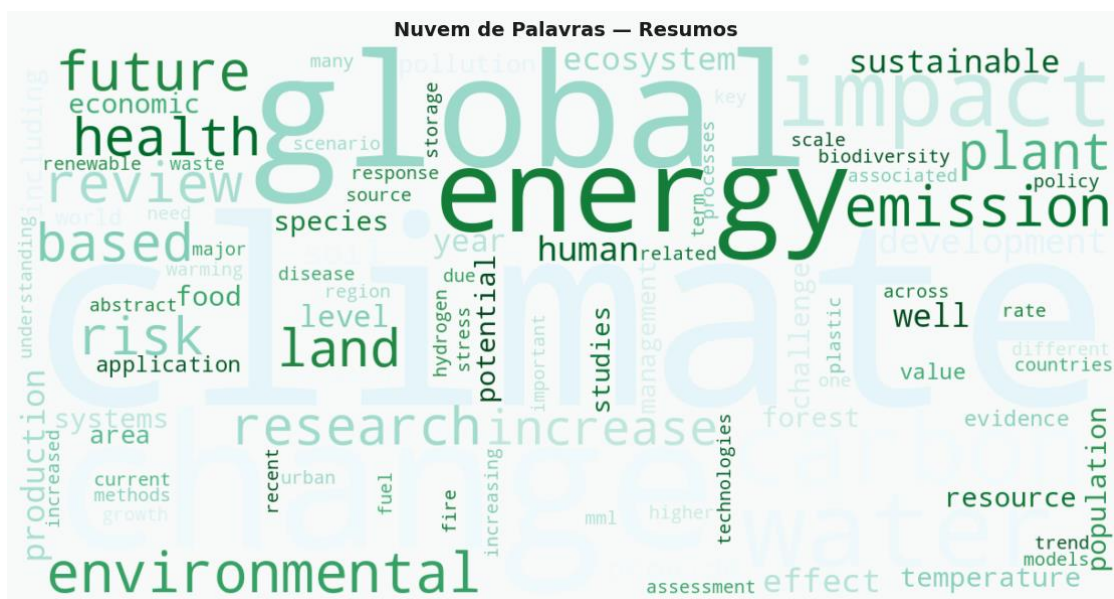
As nuvens de palavras também evidenciam a efetividade da etapa de filtragem temática, uma vez que os termos predominantes permanecem alinhados ao domínio de sustentabilidade e ciências ambientais, reduzindo a presença de conteúdos pouco relacionados ao escopo definido para o sistema de recomendação (Figuras 3 e 4).

Figura 4. Nuvem de palavras gerada a partir dos títulos dos artigos do *corpus* filtrado.



Fonte: Autoria própria.

Figura 5. Nuvem de palavras gerada a partir dos resumos dos artigos do *corpus* filtrado.



Fonte: Autoria própria.

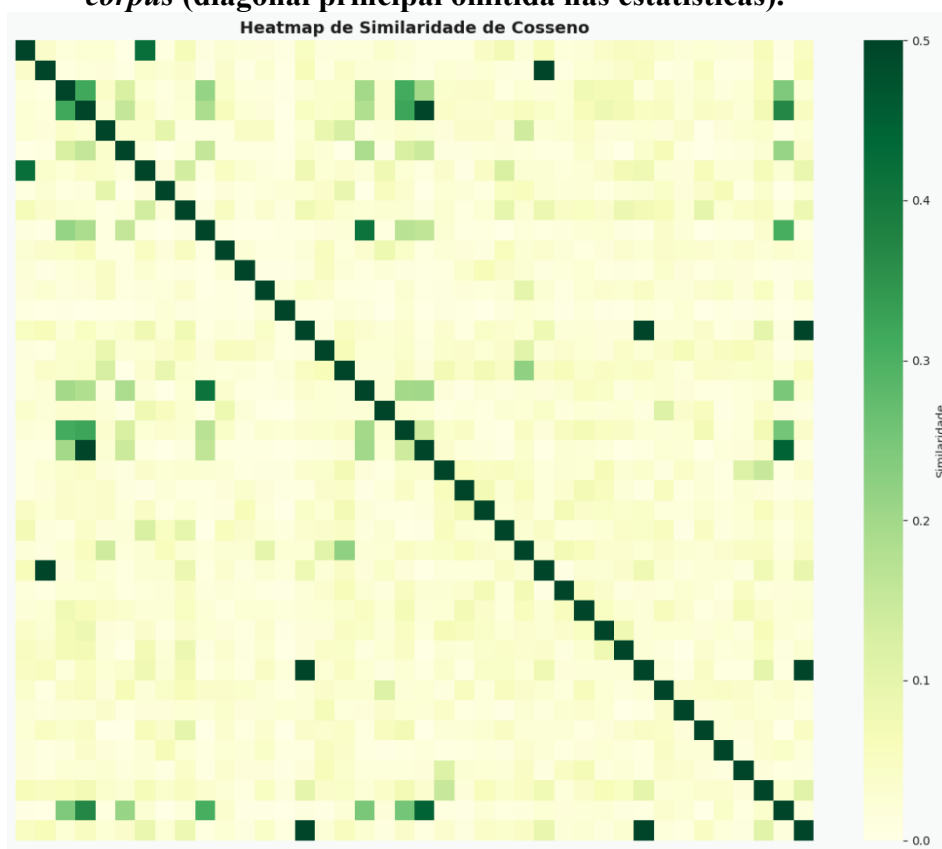
4.3 Análise da Matriz de Similaridade

A matriz de similaridade de cosseno foi computada sobre a representação TF-IDF normalizada do corpus, permitindo avaliar o grau de proximidade semântica entre os artigos selecionados. O *heatmap* gerado a partir de uma amostra dos documentos evidencia predominância de baixos níveis de similaridade entre a maior parte dos pares, representados pelas regiões mais claras da matriz, o que sugere um corpus relativamente heterogêneo em termos temáticos.

Apesar disso, observam-se agrupamentos pontuais com coloração mais intensa, indicando a existência de subconjuntos de artigos semanticamente mais próximos, especialmente em temas relacionados a energia renovável, emissões de carbono, biodiversidade e mudanças climáticas. Esse comportamento é coerente com o valor médio de similaridade obtido para o *corpus* (0,0333), indicando que, embora os documentos compartilhem um eixo temático geral ligado à sustentabilidade, existe diversidade significativa entre os conteúdos analisados.

A diagonal principal da matriz representa a similaridade de cada artigo consigo mesmo, assumindo valor máximo, enquanto os demais elementos refletem a proximidade relativa entre diferentes documentos do corpus (Figura 5).

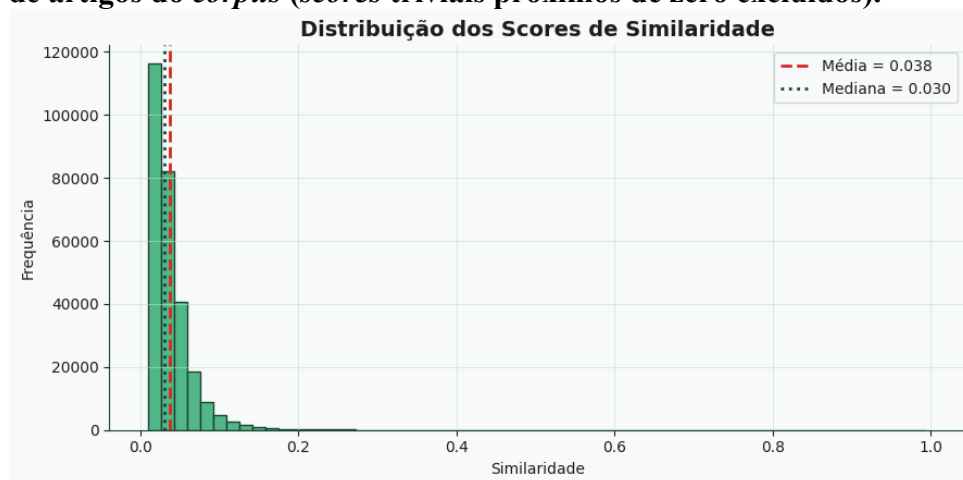
Figura 5. Heatmap de similaridade de cosseno para os primeiros 40 artigos do *corpus* (diagonal principal omitida nas estatísticas).



Fonte: Autoria própria.

A distribuição geral dos scores de similaridade, computada sobre todos os pares do *corpus* excluindo a diagonal, apresenta concentração em valores baixos, o que é esperado em corpora com diversidade temática. A média e a mediana dos scores são indicadas por linhas de referência no histograma, permitindo caracterizar a densidade da similaridade no *corpus* como um todo (Figura 6).

Figura 6. Distribuição dos scores de similaridade de cosseno entre todos os pares de artigos do corpus (scores triviais próximos de zero excluídos).



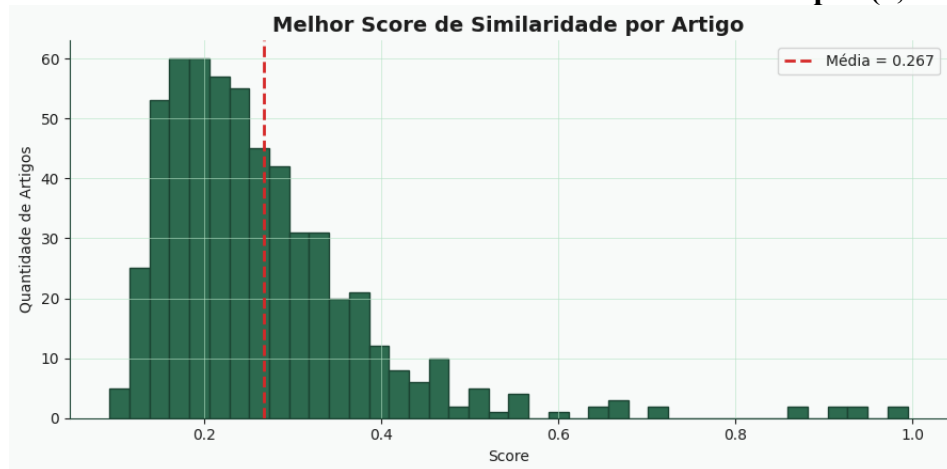
Fonte: Autoria própria.

A análise do melhor score de similaridade por artigo, definido como o maior valor de similaridade que cada documento apresenta em relação aos demais artigos do corpus, indica que a maior parte dos documentos possui ao menos um vizinho semanticamente próximo. A distribuição concentra-se principalmente entre os intervalos de 0,15 e 0,35, sugerindo presença de relações temáticas moderadas entre os artigos recuperados pelo sistema.

Observa-se também a existência de um número reduzido de artigos com scores mais elevados, próximos de 1,0, indicando pares altamente similares semanticamente. Por outro lado, artigos com menores valores máximos de similaridade tendem a representar conteúdos mais específicos ou subtemas menos recorrentes no corpus, o que pode reduzir a qualidade das recomendações associadas a esses documentos.

A linha tracejada vermelha representa a média dos melhores scores de similaridade por artigo, correspondente a 0,267, funcionando como referência para a distribuição geral da proximidade semântica entre os documentos do corpus (Figura 7).

Figura 7. Distribuição do melhor score de similaridade por artigo. A linha tracejada vermelha indica a média dos maiores scores de similaridade do corpus (0,267).

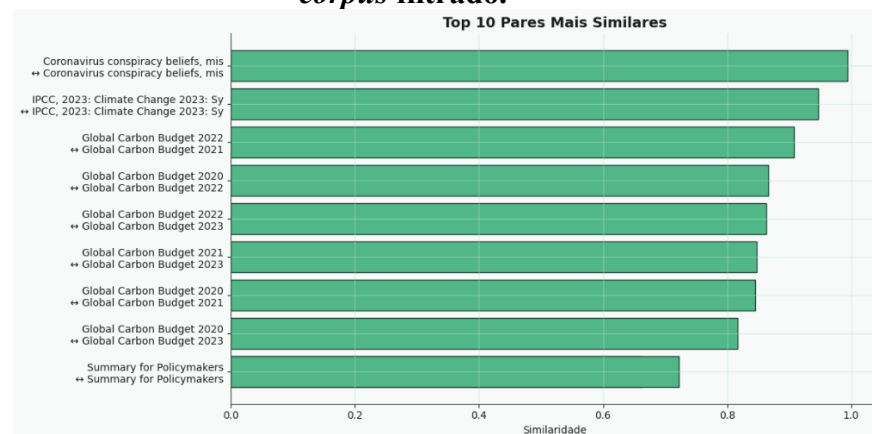


Fonte: Autoria própria.

4.4 Pares de Maior Similaridade e Coerência das Recomendações

Os dez pares de artigos com maior similaridade de cosseno no *corpus* foram inspecionados qualitativamente para verificar a coerência temática das recomendações geradas pelo sistema. Em geral, os pares de maior proximidade compartilham subtemas específicos, como acidificação dos oceanos, mitigação de emissões de carbono ou gestão de recursos hídricos, indicando que o modelo TF-IDF com ponderação de títulos captura adequadamente a proximidade lexical entre documentos com vocabulário técnico denso e específico ao domínio (Figura 8).

Figura 8. Dez pares de artigos com maior score de similaridade de cosseno no *corpus* filtrado.

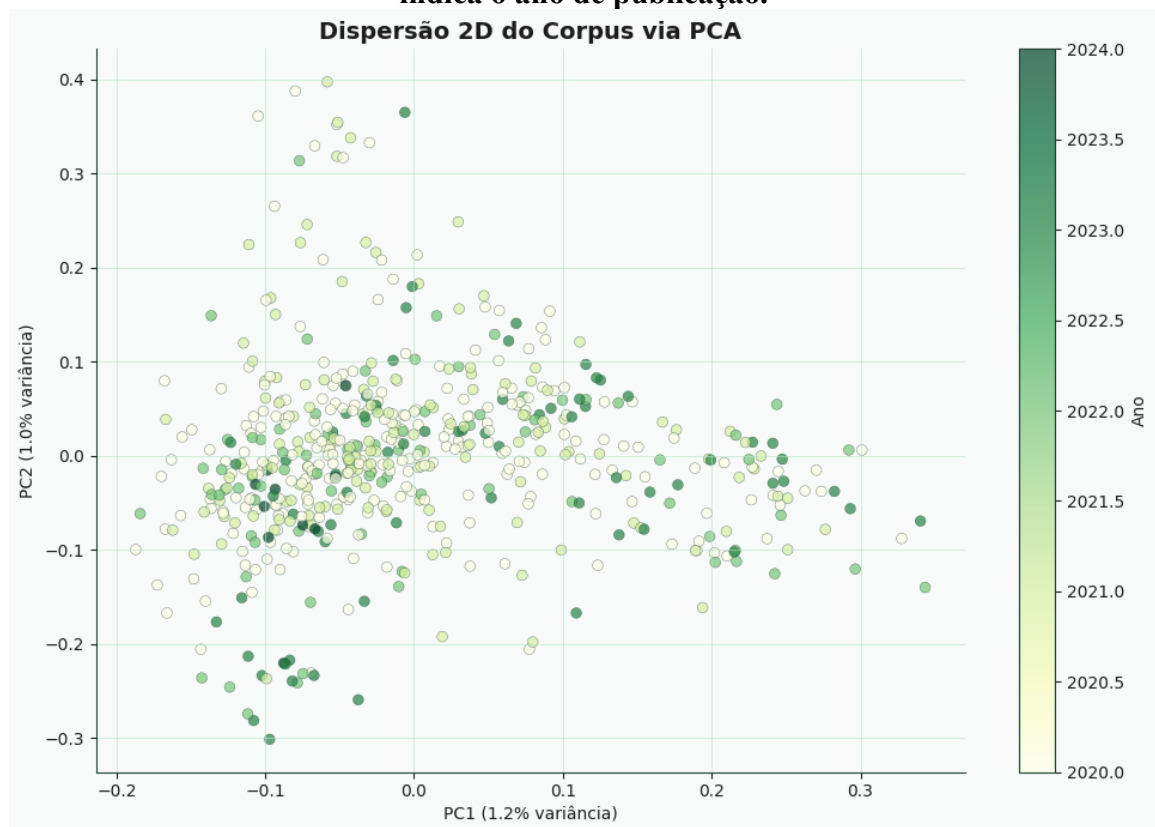


Fonte: Autoria própria.

4.5 Estrutura Semântica do *Corpus* (PCA)

A projeção bidimensional do *corpus* via Análise de Componentes Principais (PCA), aplicada sobre a matriz TF-IDF normalizada, permite visualizar a estrutura semântica global do conjunto de dados. Cada ponto representa um artigo, com a cor indicando o ano de publicação. A variância explicada pelos dois primeiros componentes é indicada nos eixos (Figura 9).

Figura 9. Dispersão 2D do *corpus* via PCA. Cada ponto representa um artigo; a cor indica o ano de publicação.



Fonte: Autoria própria.

A presença de agrupamentos visíveis na projeção indica que o *corpus* apresenta estrutura semântica interna, com subconjuntos de artigos lexicalmente coesos. Artigos distantes do núcleo central do gráfico tendem a pertencer a subtemas específicos com vocabulário pouco compartilhado com o restante do *corpus*, condição que, conforme discutido na Seção 4.3, está associada a scores de similaridade mais baixos e, conseqüentemente, a recomendações de menor qualidade.

4.6 Avaliação Qualitativa das Recomendações

Os testes qualitativos do sistema foram conduzidos por meio de diferentes estratégias de entrada, incluindo busca por índice e busca por título parcial. Os resultados variaram em função da representatividade temática do artigo de referência dentro do corpus.

Quando o artigo de entrada apresenta maior centralidade temática, isto é, quando compartilha vocabulário com um número significativo de documentos do conjunto de dados, as recomendações geradas demonstram maior coerência semântica. Esse comportamento foi observado nas consultas associadas a temas como *renewable* e *biodiversity*, para as quais o sistema recuperou artigos relacionados a transição energética, emissões de carbono, biodiversidade e recursos naturais.

Em contrapartida, artigos com vocabulário altamente específico ou associados a subtemas menos recorrentes tendem a produzir recomendações menos consistentes. Esse comportamento é esperado em abordagens baseadas em TF-IDF com similaridade de cosseno, uma vez que o método depende diretamente da sobreposição lexical entre documentos, apresentando limitações na identificação de relações semânticas latentes entre textos com terminologias distintas, ainda que tematicamente relacionados.

5 DISCUSSÃO

5.1 Eficácia e Limites do Modelo de Espaço Vetorial

A abordagem baseada em TF-IDF com similaridade de cosseno constitui uma das técnicas mais consolidadas em sistemas de recuperação da informação e filtragem por conteúdo. Manning et al. (2009) formalizam o modelo de espaço vetorial com ponderação TF-IDF como fundamento para recuperação de documentos por relevância léxica, demonstrando sua robustez em corpora com vocabulário técnico especializado. Lops *et al.* (2011) destacam que sistemas de recomendação baseados em conteúdo que utilizam TF-IDF e similaridade de cosseno apresentam desempenho satisfatório quando os itens compartilham terminologia característica do domínio, embora tendam a perder precisão em corpora heterogêneos e multidisciplinares. Revisões mais recentes corroboram essa caracterização: Kreutz e Schenkel (2022), em levantamento abrangente sobre sistemas de recomendação de artigos científicos publicados entre 2019 e 2021, apontam que métodos fundamentados predominantemente em correspondência léxica enfrentam limitações à medida que cresce a diversidade temática dos conjuntos de dados, indicando o uso de representações semânticas densas como uma das principais direções de evolução na área.

Os resultados obtidos neste trabalho mostram-se consistentes com essas observações. O sistema apresentou melhor desempenho em consultas associadas a subtemas amplamente representados no corpus, como energia renovável, biodiversidade, mudanças climáticas e emissões de carbono, nos quais existe maior compartilhamento de vocabulário entre os documentos. Nessas situações, as recomendações produzidas demonstraram coerência temática satisfatória, recuperando artigos semanticamente próximos ao documento de referência.

Esse comportamento evidencia uma limitação inerente ao modelo TF-IDF, cuja capacidade de recomendação depende diretamente da sobreposição lexical entre os textos, dificultando a identificação de relações semânticas mais profundas entre documentos que

utilizam terminologias distintas para tratar de tópicos relacionados (Deerwester *et al.*, 1990). Como apontam Mikolov et al. (2013), modelos puramente estatísticos e baseados em abordagens *bag-of-words* falham em capturar o contexto e a sinonímia, demandando representações vetoriais densas para mitigar a heterogeneidade do corpus encontrada nesta análise (onde a similaridade média foi de apenas 0,0333).

5.2 Ponderação de Campos e Estratégias de Pré-processamento

A triplicação do peso do campo título na representação TF-IDF e a expansão das *stopwords* customizadas foram implementadas com o objetivo de aumentar a especificidade das representações documentais. Essas escolhas se fundamentam em evidências empíricas da literatura: Manning, Raghavan e Schütze (2008) discutem que a ponderação diferenciada de campos, atribuindo maior relevância a metadados mais informativos como títulos, pode melhorar a discriminação entre documentos em tarefas de recuperação por similaridade. Lops *et al.* (2011) destacam que a remoção de termos de baixo valor discriminativo é uma etapa crítica no pré-processamento para sistemas de recomendação baseados em conteúdo, dado que termos genéricos inflacionam artificialmente a similaridade entre documentos de domínios distintos.

Apesar de contribuírem para o refinamento das representações, essas estratégias revelaram-se insuficientes para superar as limitações intrínsecas ao modelo de espaço vetorial quando aplicado a corpora heterogêneos. O TF-IDF opera sobre a correspondência lexical entre termos, sem capacidade de capturar relações semânticas latentes entre documentos que tratam de tópicos relacionados por meio de vocabulários distintos. Essa limitação estrutural não é sanada por ajustes de ponderação ou filtragem de *stopwords*, conforme evidenciado pelos experimentos reportados neste trabalho e corroborado pela literatura recente sobre sistemas de recomendação de artigos científicos (Kreutz; Schenkel, 2022).

5.3 Limitações do Modelo e Necessidade de Representações Semânticas

A principal limitação identificada diz respeito à incapacidade do modelo TF-IDF de capturar similaridade semântica entre documentos lexicalmente distintos, mas tematicamente próximos. Reimers e Gurevych (2019) introduziram o Sentence-BERT (SBERT), uma modificação da arquitetura BERT que emprega redes siamesas e estruturas de *triplet network* para gerar *embeddings* de sentenças semanticamente significativos comparáveis por similaridade de cosseno, reduzindo o esforço computacional de 65 horas para aproximadamente 5 segundos na identificação do par mais similar em uma coleção de 10.000 sentenças, mantendo a acurácia do modelo original. A superioridade de abordagens baseadas em *embeddings* contextuais sobre métodos de correspondência léxica em tarefas de similaridade semântica entre documentos científicos foi mais recentemente demonstrada por Putra *et al.* (2025), que aplicaram *Sentence Transformers* com dados do arXiv e obtiveram resultados superiores aos de sistemas baseados em TF-IDF isolado, especialmente em consultas que envolvem relações conceituais não expressas por termos idênticos. Nessa mesma direção, um *pipeline* automatizado para revisão de literatura avaliado por Javaji *et al.* (2023) comparou diretamente TF-IDF com modelos de *embeddings* como o all-MiniLM-L6-v2, concluindo que o TF-IDF apresentou dificuldades em capturar significados semânticos mais profundos, enquanto os modelos baseados em *transformers* forneceram cobertura conceitual mais abrangente. A adoção de modelos como o SBERT representaria um avanço significativo para o presente sistema, permitindo identificar proximidade temática entre artigos que compartilham conceitos, mas não necessariamente termos idênticos, cenário frequente em corpora científicos multidisciplinares como o de sustentabilidade ambiental.

5.4 Avaliação Quantitativa e Trabalhos Futuros

A avaliação do sistema foi conduzida de forma predominantemente qualitativa, por meio da inspeção dos títulos retornados e da análise da distribuição dos scores de similaridade.

A ausência de avaliação quantitativa com métricas padronizadas de recuperação de informação constitui uma limitação metodológica relevante. Järvelin e Kekäläinen (2002) propuseram o *Normalized Discounted Cumulative Gain* (NDCG), métrica amplamente adotada em sistemas de recuperação de informação e recomendação por capturar tanto a relevância dos itens retornados quanto sua posição no ranking, penalizando logaritmicamente itens relevantes posicionados mais abaixo na lista. Complementarmente, a Precision@K mensura a proporção de itens relevantes entre os K primeiros retornados pelo sistema, sendo especialmente adequada para avaliar sistemas nos quais o usuário interage prioritariamente com os resultados no topo da lista (Manning *et al.*, 2009). A aplicação combinada dessas métricas é destacada por Kreutz e Schenkel (2022) como prática padrão em avaliações rigorosas de sistemas de recomendação de artigos científicos, sendo essencial para comparações reprodutíveis entre abordagens distintas.

A implementação dessas métricas em trabalhos futuros requereria a construção de um conjunto de avaliação com julgamentos de relevância anotados, tarefa viável mediante amostragem do *corpus* e anotação manual ou semi-automática com apoio de especialistas no domínio. Como desdobramentos prioritários, recomenda-se: a substituição ou complementação do TF-IDF por modelos de *embeddings* como o Sentence-BERT (Reimers & Gurevych, 2019) ou variantes especializadas em textos científicos; a adoção de critérios explícitos de centralidade temática na seleção do artigo de referência durante os testes, priorizando documentos com maior densidade de conexões no grafo de similaridade do *corpus*; e a implementação de avaliação quantitativa com Precision@K e NDCG para mensuração objetiva e reprodutível do desempenho do sistema.

6 CONCLUSÃO

Este trabalho apresentou o desenvolvimento de um sistema de recomendação de artigos científicos sobre sustentabilidade, baseado em filtragem por conteúdo com representação tf-idf e similaridade de cosseno, aplicado a um corpus de 569 artigos coletados via api do openalex. Os resultados obtidos demonstraram a viabilidade da abordagem como prova de conceito para recomendação de literatura científica em ciências ambientais e sustentabilidade.

As análises realizadas indicaram que o sistema foi capaz de recuperar artigos semanticamente coerentes em subtemas bem representados no corpus, especialmente em tópicos relacionados a mudanças climáticas, energia renovável, biodiversidade e emissões de carbono. A etapa de filtragem temática mostrou-se efetiva na delimitação do escopo do conjunto de dados, contribuindo para a construção de um corpus alinhado aos objetivos da pesquisa.

Por outro lado, os resultados também evidenciaram limitações inerentes ao modelo de espaço vetorial baseado em tf-idf, sobretudo em artigos associados a subtemas menos recorrentes ou com vocabulário altamente específico. Nessas situações, a dependência da sobreposição lexical reduz a capacidade do sistema de capturar relações semânticas mais profundas entre os documentos.

Como trabalhos futuros, destacam-se a incorporação de embeddings semânticos contextuais, baseados em modelos de linguagem, a aplicação de técnicas híbridas de recomendação e a realização de avaliações quantitativas com métricas padronizadas, visando aumentar a robustez e a qualidade das recomendações produzidas pelo sistema.

REFERÊNCIAS

- ACRL – ASSOCIATION OF COLLEGE AND RESEARCH LIBRARIES. Framework for Information Literacy for Higher Education. Chicago, 2016.
- AGGARWAL, C. C. An introduction to recommender systems. In: RICCI, F.; ROKACH, L.; SHAPIRA, B. (Eds.). Recommender systems: the textbook. Cham: Springer International Publishing, 2016. p. 1–28.
- ARXIV. arXiv e-Print archive. Cornell University, 2024. Disponível em: <https://arxiv.org/>. Acesso em: 30 mar. 2026.
- BAEZA-YATES, R.; RIBEIRO-NETO, B. Modern information retrieval: the concepts and technology behind search. 2. ed. [S. l.]: Addison-Wesley Professional, 2011.
- BAWDEN, D.; ROBINSON, L. The dark side of information: overload, anxiety and other paradoxes. *Journal of Information Science*, v. 35, n. 2, p. 180–191, 2009.
- BEEL, J.; GIPP, B.; LANGER, S.; BREITINGER, C. Research-paper recommender systems: a literature survey. *International Journal on Digital Libraries*, [S. l.], v. 17, n. 4, p. 305–338, 2016.
- BISONG, E. Building machine learning and deep learning models on Google Cloud Platform. Berkeley: Apress, 2019.
- BORNMAN, L.; MUTZ, R. Growth rates of modern science: A bibliometric analysis. *Journal of the Association for Information Science and Technology*, v. 66, n. 11, p. 2215–2222, 2015.
- DEERWESTER, S. *et al.* Indexing by latent semantic analysis. *Journal of the American Society for Information Science*, v. 41, n. 6, p. 391–407, 1990.
- DRACHSER, H.; VERBERT, K.; SANTOS, O. C.; MANOUSELIS, N. Panorama of recommender systems to support learning. In: RICCI, F.; ROKACH, L.; SHAPIRA, B. (Eds.). *Recommender systems handbook*. Boston: Springer, 2015. p. 421–451.

- EPPLER, M. J.; MENGIS, J. The concept of information overload: A review of literature. *The Information Society*, v. 20, n. 5, p. 325–344, 2004.
- HEAD, A. J.; EISENBERG, M. B. How college students use the web to conduct everyday life research. *Project Information Literacy*, 2010.
- IPCC. *Climate Change 2023: Synthesis Report*. Geneva: Intergovernmental Panel on Climate Change, 2023.
- JÄRVELIN, K.; KEKÄLÄINEN, J. Cumulated gain-based evaluation of IR techniques. *ACM Transactions on Information Systems*, v. 20, n. 4, p. 422-446, 2002.
- JURAFSKY, D.; MARTIN, J. H. *Speech and Language Processing: An Introduction to Natural Language Processing, Computational Linguistics, and Speech Recognition with Language Models*. 3. ed. (rascunho online em elaboração). Stanford: Stanford University, 2024. Disponível em: <https://web.stanford.edu/~jurafsky/slp3/>. Acesso em: 20 mar. 2026.
- KREUTZ, C. K.; SCHENKEL, R. Scientific paper recommendation systems: a literature review of recent publications. *International Journal on Digital Libraries*, v. 23, n. 4, p. 335-369, 2022. DOI: 10.1007/s00799-022-00339-w.
- LOPS, P.; DE GEMMIS, M.; SEMERARO, G. Content-based recommender systems: State of the art and trends. In: RICCI, F.; ROKACH, L.; SHAPIRA, B. (Eds.). *Recommender Systems Handbook*. 3. ed. Cham: Springer, 2010. p. 73–105.
- MANNING, C. D.; RAGHAVAN, P.; SCHÜTZE, H. *Introduction to Information Retrieval*. Cambridge: Cambridge University Press, 2009.
- MANOUSELIS, N.; DRACHSER, H.; VERBERT, K.; DUVAL, E. *Recommender systems for learning*. New York: Springer, 2013.

- MIKOLOV, Tomas *et al.* Distributed representations of words and phrases and their compositionality. *In: Advances in Neural Information Processing Systems (NeurIPS)*, v. 26, p. 3111-3119, 2013.
- ONU – ORGANIZAÇÃO DAS NAÇÕES UNIDAS. Transforming our world: the 2030 Agenda for Sustainable Development. New York, 2015.
- PEDREGOSA, F. *et al.* Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, v. 12, p. 2825–2830, 2011.
- PRIEM, J.; PIWOWAR, H.; ORR, R. OpenAlex: a fully-open index of scholarly works, authors, venues, institutions, and concepts. *arXiv preprint, arXiv:2205.01833*, 2022.
- PUTRA, A. P.; SINGGIH PUTRI, D. P.; WIRANATHA, A. C. Scientific paper recommendation system: application of sentence transformers and cosine similarity using arXiv data. *Journal of Applied Informatics and Computing*, 2025.
- RAMOS, J. Using TF-IDF to determine word relevance in document queries. *In: Proceedings of the First Instructional Conference on Machine Learning*. 2003. p. 29–48.
- REIMERS, N.; GUREVYCH, I. Sentence-BERT: sentence embeddings using Siamese BERT-networks. *In: Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP-IJCNLP)*. Hong Kong: ACL, 2019. p. 3982-3992. DOI: 10.18653/v1/D19-1410.
- RESNICK, P.; VARIAN, H. R. Recommender systems. *Communications of the ACM*, v. 40, n. 3, p. 56–58, 1997.
- RICCI, F.; ROKACH, L.; SHAPIRA, B. (Eds.). *Recommender Systems Handbook*. 2. ed. New York: Springer, 2015.
- RICCI, F.; ROKACH, L.; SHAPIRA, B. Recommender systems: Techniques, applications, and challenges. *In: RICCI, F.; ROKACH, L.; SHAPIRA, B. (Eds.). Recommender Systems Handbook*. 3. ed. Cham: Springer, 2022. p. 1–35.

- ROKNI, M. *et al.* Scientific paper recommender system using deep learning and link prediction in citation network. *Heliyon*, v. 10, n. 14, 2024. Disponível em: <https://www.sciencedirect.com/science/article/pii/S2405844024107165>.
- SALTON, G.; BUCKLEY, C. Term-weighting approaches in automatic text retrieval. *Information Processing & Management*, v. 24, n. 5, p. 513–523, 1988.
- TAN, P.-N.; STEINBACH, M.; KUMAR, V. *Introduction to Data Mining*. Boston: Addison-Wesley, 2006.
- UNESCO. *Reimagining our futures together: a new social contract for education*. Paris: UNESCO, 2021.

ANEXO A – Código-fonte do Sistema de Recomendação

A seguir, apresenta-se o código-fonte completo do sistema de recomendação desenvolvido em Python, documentado com comentários explicativos referenciando as etapas metodológicas descritas ao longo deste trabalho.

```
# =====
# SISTEMA DE RECOMENDAÇÃO DE ARTIGOS CIENTÍFICOS - OpenAlex
# Metodologia: Filtragem por Conteúdo com TF-IDF + Cosseno
# Versão: 3.0
# Ambiente: Google Colab
# =====

# =====
# INSTALAÇÃO (execute antes se necessário)
# =====
# !pip install requests scikit-learn pandas numpy

import requests
import pandas as pd
import numpy as np
import re
import time

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import normalize

# =====
# ETAPA 1 – FUNÇÃO PARA RECONSTRUIR ABSTRACT
# =====

def reconstruir_abstract(inverted_index):

    if not inverted_index or not isinstance(inverted_index, dict):
        return ""

    pares = []

    for palavra, posicoes in inverted_index.items():
        for pos in posicoes:
            pares.append((pos, palavra))

    pares.sort(key=lambda x: x[0])

    return " ".join(p for _, p in pares)

# =====
# ETAPA 2 – COLETA DE DADOS OPENALEX
# =====

def coletar_artigos_openalex(
    conceitos,
    anos=(2020, 2024),
    meta_total=1000
):
```



```

BASE_URL = "https://api.openalex.org/works"

MAILTO = "slthaina@outlook.com"

registros = []
ids_vistos = set()

print("=" * 70)
print("INICIANDO COLETA")
print("=" * 70)

print(f"Meta: {meta_total} artigos")
print(f"Período: {anos[0]}-{anos[1]}")
print(f"Total de conceitos: {len(conceitos)}\n")

for conceito in conceitos:

    print(f"→ Coletando: {conceito}")

    cursor = "*"

    paginas = 0
    artigos_conceito = 0

    while True:

        params = {

            "search": conceito,

            "filter": (
                f"from_publication_date:{anos[0]}-01-01,"
                f"to_publication_date:{anos[1]}-12-31,"
                "has_abstract:true"
            ),

            "select": (
                "id,"
                "title,"
                "abstract_inverted_index,"
                "publication_year,"
                "authorships,"
                "primary_location,"
                "concepts"
            ),

            "sort": "cited_by_count:desc",

            "per-page": 200,

            "cursor": cursor,

            "mailto": MAILTO,
        }

        try:

            resposta = requests.get(
                BASE_URL,
                params=params,
                timeout=30

```

```

    )

    resposta.raise_for_status()

    dados = resposta.json()

except requests.exceptions.RequestException as e:

    print(f"[ERRO] {e}")
    break

resultados = dados.get("results", [])

if not resultados:
    break

for obra in resultados:

    obra_id = obra.get("id", "")

    if obra_id in ids_vistos:
        continue

    ids_vistos.add(obra_id)

    titulo = obra.get("title") or ""

    abstract = reconstruir_abstract(
        obra.get("abstract_inverted_index")
    )

    if not titulo.strip():
        continue

    if not abstract.strip():
        continue

    ano = obra.get("publication_year")

    autores = ", ".join(
        a.get("author", {}).get("display_name", "")
        for a in (obra.get("authorships") or [])
        if a.get("author", {}).get("display_name")
    )

    conceitos_obra = ", ".join(
        c.get("display_name", "")
        for c in (obra.get("concepts") or [])[:5]
    )

    loc = obra.get("primary_location") or {}

    doi = (
        loc.get("landing_page_url")
        or obra_id
    )

    registros.append({

        "titulo": titulo,
        "autores": autores,

```

```

        "resumo": abstract,
        "ano": ano,
        "openalex_id": obra_id,
        "doi": doi,
        "conceitos_api": conceitos_obra,
    })

    artigos_conceito += 1

    paginas += 1

    if paginas % 2 == 0:

        print(
            f"Página {paginas} | "
            f"Total acumulado: {len(registros)}"
        )

        if len(registros) >= meta_total:
            break

        meta = dados.get("meta", {})

        cursor = meta.get("next_cursor")

        if not cursor:
            break

        time.sleep(0.5)

    print(f"✓ Artigos coletados: {artigos_conceito}\n")

    if len(registros) >= meta_total:
        print("✓ Meta atingida.\n")
        break

colunas = [
    "titulo",
    "autores",
    "resumo",
    "ano",
    "openalex_id",
    "doi",
    "conceitos_api",
]

df = pd.DataFrame(registros, columns=colunas)

df = df.head(meta_total)

print("=" * 70)
print(f"✓ TOTAL FINAL: {len(df)} artigos")
print("=" * 70)

return df

# =====
# CONFIGURAÇÕES
# =====

```

```

CONCEITOS_OPENALEX = [

    "Climate change",
    "Sustainability",
    "Environmental science",
    "Renewable energy",
    "Biodiversity",
    "Deforestation",
    "Greenhouse gas",
    "Ocean acidification",
    "Water scarcity",
    "Wildfire",

]

ANO_INICIO = 2020
ANO_FIM = 2024

META_TOTAL = 1000

# =====
# EXECUÇÃO DA COLETA
# =====

df_raw = coletar_artigos_openalex(
    CONCEITOS_OPENALEX,
    anos=(ANO_INICIO, ANO_FIM),
    meta_total=META_TOTAL
)

# =====
# VERIFICAÇÃO
# =====

if df_raw.empty:

    raise ValueError(
        "\n[ERRO] Nenhum artigo foi coletado."
    )

# =====
# SALVAR DATASET BRUTO
# =====

df_raw.to_csv(
    "dataset_bruto_openalex.csv",
    index=False
)

print("\n✓ Dataset bruto salvo.")

# =====
# ETAPA 3 – PRÉ-PROCESSAMENTO
# =====

STOPWORDS_EN = {

    "a", "an", "the", "and", "or", "but", "if", "in", "on", "at", "to", "for", "of",
    "with", "by", "from", "as", "is", "was", "are", "were", "be", "been", "being",
    "have", "has", "had", "do", "does", "did", "will", "would", "could", "should",
    "may", "might", "shall", "can", "not", "no", "nor", "so", "yet", "both",

```

```

"either", "neither", "each", "few", "more", "most", "other", "some",
"such", "than", "then", "that", "this", "these", "those", "which", "who",
"whom", "what", "when", "where", "why", "how", "all", "any", "he", "she",
"it", "we", "they", "i", "you", "his", "her", "its", "our", "their", "my",
"your", "up", "out", "about", "into", "through", "during", "before",
"after", "above", "below", "between", "among", "while", "although",
"because", "since", "there", "here", "very", "just", "also", "only",
"over", "under", "again", "further", "once", "per", "via"
}

STOPWORDS_CUSTOM = {

    "model", "method", "study", "analysis", "approach",
    "result", "results", "paper", "proposed", "using",
    "used", "use", "show", "shown", "shows", "present",
    "presented", "data", "dataset", "performance",
    "system", "framework", "task", "tasks", "work",
    "works", "new", "novel", "evaluation", "experiment",
    "experiments", "baseline", "large", "high", "low"
}

TODAS_STOPWORDS = STOPWORDS_EN | STOPWORDS_CUSTOM

def preprocessor_texto(texto):

    if not isinstance(texto, str):
        return ""

    texto = texto.lower()

    texto = re.sub(r"[^a-z\s]", " ", texto)

    tokens = texto.split()

    tokens = [
        t for t in tokens
        if t not in TODAS_STOPWORDS
        and len(t) >= 3
    ]

    return " ".join(tokens)

print("\n" + "=" * 70)
print("PRÉ-PROCESSAMENTO")
print("=" * 70)

df = df_raw.dropna(
    subset=["titulo", "resumo"]
).copy()

print(f"✓ Registros válidos: {len(df)}")

df["titulo_limpo"] = df["titulo"].apply(
    preprocessor_texto
)

df["resumo_limpo"] = df["resumo"].apply(
    preprocessor_texto
)

```

```

print("✓ Pré-processamento concluído.")

# =====
# ETAPA 4 – FILTRAGEM TEMÁTICA
# =====

TERMOS_FORTES = {

    "climate change",
    "global warming",
    "carbon emissions",
    "greenhouse gas",
    "biodiversity loss",
    "deforestation",
    "ocean acidification",
    "wildfire",
    "water scarcity",
    "renewable energy",
}

TERMOS_MEDIOS = {

    "environment",
    "environmental",
    "energy",
    "ecosystem",
    "ecological",
    "pollution",
    "sustainability",
    "renewable",
    "conservation",
    "biodiversity",
    "forest",
    "marine",
    "climate",
}

def eh_relevante(titulo, resumo):

    texto = f"{titulo} {resumo}".lower()

    for t in TERMOS_FORTES:
        if t in texto:
            return True

    contagem_media = sum(
        1 for t in TERMOS_MEDIOS
        if t in texto
    )

    if contagem_media >= 2:
        return True

    return False

print("\n" + "=" * 70)
print("FILTRAGEM TEMÁTICA")
print("=" * 70)

df["relevante"] = df.apply(

```

```

        lambda r: eh_relevante(
            r["título"],
            r["resumo"]
        ),

        axis=1
    )

df_filtrado = df[
    df["relevante"]
].copy().reset_index(drop=True)

print(f"✓ Antes da filtragem: {len(df)}")
print(f"✓ Após filtragem: {len(df_filtrado)}")

if df_filtrado.empty:
    raise ValueError(
        "[ERRO] Dataset vazio após filtragem."
    )

print("\nDistribuição por ano:\n")

dist_ano = (
    df_filtrado["ano"]
    .value_counts()
    .sort_index()
)

for ano, qtd in dist_ano.items():

    barra = "█" * (qtd // 5)

    print(f"{int(ano)}: {qtd:>4} {barra}")

# =====
# ETAPA 5 – TF-IDF
# =====

def construir_texto_ponderado(
    titulo_limpo,
    resumo_limpo,
    peso_titulo=3
):

    titulo_rep = " ".join(
        [titulo_limpo] * peso_titulo
    )

    return f"{titulo_rep} {resumo_limpo}"

print("\n" + "=" * 70)
print("TF-IDF")
print("=" * 70)

df_filtrado["texto_final"] = df_filtrado.apply(

    lambda r: construir_texto_ponderado(
        r["titulo_limpo"],

```

```

        r["resumo_limpo"]
    ),

    axis=1
)

vectorizer = TfidfVectorizer(

    max_features=5000,
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True,
)

matriz_tfidf = vectorizer.fit_transform(
    df_filtrado["texto_final"]
)

matriz_tfidf_norm = normalize(
    matriz_tfidf,
    norm="l2"
)

print(
    f"✓ Matriz TF-IDF: "
    f"{matriz_tfidf_norm.shape[0]} artigos × "
    f"{matriz_tfidf_norm.shape[1]} features"
)

# =====
# ETAPA 6 – SIMILARIDADE
# =====

print("\n" + "=" * 70)
print("SIMILARIDADE DE COSSENO")
print("=" * 70)

matriz_similaridade = cosine_similarity(
    matriz_tfidf_norm
)

print(
    f"✓ Matriz: "
    f"{matriz_similaridade.shape}"
)

sim_sem_diagonal = matriz_similaridade[
    ~np.eye(
        matriz_similaridade.shape[0],
        dtype=bool
    )
]

print(
    f"✓ Similaridade média: "
    f"{sim_sem_diagonal.mean():.4f}"
)

# =====
# ETAPA 7 – RECOMENDAÇÃO

```



```
# =====

SCORE_MINIMO = 0.10

def recomendar_por_indice(
    indice,
    top_n=5
):

    scores = list(
        enumerate(
            matriz_similaridade[indice]
        )
    )

    scores = sorted(
        scores,
        key=lambda x: x[1],
        reverse=True
    )

    scores = [
        (i, s)

        for i, s in scores

        if i != indice
        and s >= SCORE_MINIMO
   ][:top_n]

    recomendacoes = []

    for idx, score in scores:

        recomendacoes.append({

            "titulo":
                df_filtrado.iloc[idx]["titulo"],

            "similaridade":
                round(score, 4),

            "ano":
                df_filtrado.iloc[idx]["ano"],
        })

    return pd.DataFrame(recomendacoes)

def recomendar_por_titulo(
    titulo_parcial,
    top_n=5
):

    mascara = (
        df_filtrado["titulo"]
        .str.lower()
        .str.contains(
            titulo_parcial.lower(),
            na=False

```

```

    )
)

encontrados = df_filtrado[mascara]

if encontrados.empty:

    print(
        f"Nenhum artigo encontrado "
        f"para '{titulo_parcial}'."
    )

    return pd.DataFrame()

indice = encontrados.index[0]

print(
    f"\nArtigo base:\n"
    f"{df_filtrado.iloc[indice]['titulo']}\n"
)

return recomendar_por_indice(
    indice,
    top_n
)

# =====
# TESTES
# =====

print("\n" + "=" * 70)
print("TESTES")
print("=" * 70)

print("\n[ Recomendações para 'renewable' ]")

rec = recomendar_por_titulo(
    "renewable",
    top_n=5
)

if not rec.empty:
    print(rec.to_string(index=False))

print("\n[ Recomendações para 'biodiversity' ]")

rec2 = recomendar_por_titulo(
    "biodiversity",
    top_n=5
)

if not rec2.empty:
    print(rec2.to_string(index=False))

# =====
# SALVAR DATASET FINAL
# =====

df_filtrado.to_csv(
    "dataset_filtrado_openalex.csv",
    index=False
)

```

```
)

print("\n✓ Dataset filtrado salvo.")

print("\n" + "=" * 70)
print("PIPELINE CONCLUÍDO COM SUCESSO")
print("=" * 70)

print(f"✓ Artigos finais: {len(df_filtrado)}")

print(
    f"✓ Features TF-IDF: "
    f"{matriz_tfidf_norm.shape[1]}"
)

print(
    f"✓ Matriz similaridade: "
    f"{matriz_similaridade.shape}"
)
```

ANEXO B – Código-fonte do Script de Visualização

A seguir, apresenta-se o código-fonte completo do script responsável pela geração das visualizações utilizadas na análise dos resultados, desenvolvido em Python e documentado com comentários explicativos referenciando as figuras produzidas e descritas ao longo deste trabalho.

```
# =====
# VISUALIZAÇÕES DO SISTEMA DE RECOMENDAÇÃO - OpenAlex
# Ambiente: Google Colab
# =====

# =====
# INSTALAÇÃO (execute antes se necessário)
# =====
# !pip install pandas numpy matplotlib seaborn scikit-learn wordcloud

# =====
# IMPORTS
# =====

import os
import re

import numpy as np
import pandas as pd

import matplotlib.pyplot as plt
import seaborn as sns

from collections import Counter
from wordcloud import WordCloud

from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.metrics.pairwise import cosine_similarity
from sklearn.preprocessing import normalize
from sklearn.decomposition import PCA

# =====
# ESTILO VISUAL
# =====

VERDE_ESCURO = "#1B4332"
VERDE_MEDIO = "#2D6A4F"
VERDE_CLARO = "#52B788"
VERDE_PALIDO = "#B7E4C7"

FUNDO = "#F8FAF9"
TEXTO = "#1C1C1C"
DESTAQUE = "#D62828"

plt.rcParams.update({
    "figure.facecolor": FUNDO,
    "axes.facecolor": FUNDO,
    "axes.edgecolor": VERDE_ESCURO,
```

```

    "axes.labelcolor": TEXTO,
    "xtick.color": TEXTO,
    "ytick.color": TEXTO,
    "text.color": TEXTO,
    "font.family": "DejaVu Sans",
    "axes.spines.top": False,
    "axes.spines.right": False,
    "axes.grid": True,
    "grid.color": VERDE_PALIDO,
    "grid.linewidth": 0.6,
    "grid.alpha": 0.7,
})

# =====
# PASTA DE SAÍDA
# =====

PASTA_SAIDA = "graficos_openalex"

os.makedirs(PASTA_SAIDA, exist_ok=True)

def salvar(fig, nome):

    caminho = f"{PASTA_SAIDA}/{nome}.png"

    fig.savefig(
        caminho,
        dpi=200,
        bbox_inches="tight",
        facecolor=FUNDO
    )

    print(f"✓ Salvo: {caminho}")

    plt.show()
    plt.close(fig)

# =====
# CARREGAMENTO DO DATASET
# =====

print("=" * 65)
print("CARREGANDO DATASET")
print("=" * 65)

df = pd.read_csv("dataset_filtrado_openalex.csv")

print(f"✓ Artigos carregados: {len(df)}")

# =====
# STOPWORDS
# =====

STOPWORDS_EN = {
    "a", "an", "the", "and", "or", "but", "if", "in", "on", "at", "to", "for", "of",
    "with", "by", "from", "as", "is", "was", "are", "were", "be", "been", "being",
    "have", "has", "had", "do", "does", "did", "will", "would", "could", "should",
    "may", "might", "shall", "can", "not", "no", "nor", "so", "yet", "both", "either",

```

```

"neither", "each", "few", "more", "most", "other", "some", "such", "than", "then",
"that", "this", "these", "those", "which", "who", "whom", "what", "when", "where",
    "why", "how", "all", "any", "both", "he", "she", "it", "we", "they", "i", "you",
"his", "her", "its", "our", "their", "my", "your", "up", "out", "about", "into",
"through", "during", "before", "after", "above", "below", "between", "among",
"while", "although", "because", "since", "there", "here", "very", "just", "also",
    "only", "over", "under", "again", "further", "once", "per", "via", "re", "eg",
    "ie", "etc",
}

STOPWORDS_CUSTOM = {
    "model", "method", "study", "analysis", "approach", "result", "results",
    "paper", "proposed", "based", "using", "used", "use", "show", "shown",
    "shows", "present", "presented", "presents", "data", "dataset",
    "performance", "system", "framework", "task", "tasks", "work",
    "works", "new", "novel", "existing", "state", "art", "two", "three",
    "four", "five", "one", "first", "second", "third", "evaluation",
    "experiment", "experiments", "experimental", "baseline", "large",
    "high", "low", "well", "however", "thus", "therefore", "hence",
    "fig", "table", "et", "al", "section", "propose", "provides",
    "provide", "able", "different", "various", "several", "recent",
    "current", "previous", "following", "given", "set", "sets",
    "number", "numbers", "type", "types", "level", "levels",
    "case", "cases",
}

TODAS_SW = STOPWORDS_EN | STOPWORDS_CUSTOM

# =====
# LIMPEZA TEXTUAL
# =====

def limpar(texto):
    if not isinstance(texto, str):
        return ""

    texto = texto.lower()

    texto = re.sub(r"^[a-z\s]", " ", texto)

    tokens = texto.split()

    tokens = [
        t for t in tokens
        if t not in TODAS_SW and len(t) >= 3
    ]

    return " ".join(tokens)

# =====
# GARANTE COLUNAS LIMPAS
# =====

if "titulo_limpo" not in df.columns:
    df["titulo_limpo"] = df["titulo"].apply(limpar)

```

```

if "resumo_limpo" not in df.columns:
    df["resumo_limpo"] = df["resumo"].apply(limpar)

# =====
# TEXTO FINAL
# =====

def construir_texto(row):

    titulo = row["titulo_limpo"]
    resumo = row["resumo_limpo"]

    titulo_ponderado = " ".join([titulo] * 3)

    return f"{titulo_ponderado} {resumo}"

df["texto_final"] = df.apply(construir_texto, axis=1)

# =====
# TF-IDF + SIMILARIDADE
# =====

print("\n" + "=" * 65)
print("RECONSTRUINDO TF-IDF")
print("=" * 65)

vectorizer = TfidfVectorizer(
    max_features=5000,
    ngram_range=(1, 2),
    min_df=2,
    sublinear_tf=True,
)

tfidf = vectorizer.fit_transform(df["texto_final"])

tfidf_norm = normalize(tfidf, norm="l2")

matriz_sim = cosine_similarity(tfidf_norm)

print(f"✓ TF-IDF: {tfidf_norm.shape}")
print(f"✓ Similaridade: {matriz_sim.shape}")

# =====
# [1] DISTRIBUIÇÃO POR ANO
# =====

print("\n[1] Distribuição por ano")

dist_ano = df["ano"].value_counts().sort_index()

fig, ax = plt.subplots(figsize=(8, 4.5))

bars = ax.bar(
    dist_ano.index.astype(str),
    dist_ano.values,
    color=VERDE_MEDIO,
    edgecolor=VERDE_ESCURO
)

```

```

for bar, valor in zip(bars, dist_ano.values):

    ax.text(
        bar.get_x() + bar.get_width()/2,
        bar.get_height() + 1,
        str(valor),
        ha="center",
        fontsize=10,
        fontweight="bold"
    )

ax.set_title(
    "Distribuição de Artigos por Ano",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel("Ano")
ax.set_ylabel("Número de Artigos")

fig.tight_layout()

salvar(fig, "01_distribuicao_por_ano")

# =====
# [2] TOP 20 PALAVRAS
# =====

print("[2] Top palavras")

tokens = " ".join(df["resumo_limpo"]).split()

freq = Counter(tokens).most_common(20)

palavras = [x[0] for x in freq]
contagens = [x[1] for x in freq]

fig, ax = plt.subplots(figsize=(10, 6))

bars = ax.barh(
    list(reversed(palavras)),
    list(reversed(contagens)),
    color=VERDE_CLARO,
    edgecolor=VERDE_ESCURO
)

ax.set_title(
    "Top 20 Palavras Mais Frequentes",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel("Frequência")

fig.tight_layout()

salvar(fig, "02_top20_palavras")

# =====
# [3] NUVEM DE PALAVRAS - TÍTULOS
# =====

```



```

print("[3] Wordcloud títulos")

texto_titulos = " ".join(df["titulo_limpo"])

wc_titulos = WordCloud(
    width=1200,
    height=600,
    background_color=FUNDO,
    colormap="YlGn",
    max_words=80,
    collocations=False
).generate(texto_titulos)

fig, ax = plt.subplots(figsize=(12, 6))

ax.imshow(wc_titulos, interpolation="bilinear")

ax.axis("off")

ax.set_title(
    "Nuvem de Palavras - Títulos",
    fontsize=14,
    fontweight="bold"
)

fig.tight_layout()

salvar(fig, "03_wordcloud_titulos")

# =====
# [4] NUVEM DE PALAVRAS - RESUMOS
# =====

print("[4] Wordcloud resumos")

texto_resumos = " ".join(df["resumo_limpo"])

wc_resumos = WordCloud(
    width=1200,
    height=600,
    background_color=FUNDO,
    colormap="BuGn",
    max_words=100,
    collocations=False
).generate(texto_resumos)

fig, ax = plt.subplots(figsize=(12, 6))

ax.imshow(wc_resumos, interpolation="bilinear")

ax.axis("off")

ax.set_title(
    "Nuvem de Palavras - Resumos",
    fontsize=14,
    fontweight="bold"
)

fig.tight_layout()

```

```

salvar(fig, "04_wordcloud_resumos")

# =====
# [5] HEATMAP
# =====

print("[5] Heatmap")

N = min(40, len(df))

amostra = matriz_sim[:N, :N]

fig, ax = plt.subplots(figsize=(12, 10))

sns.heatmap(
    amostra,
    cmap="YlGn",
    vmin=0,
    vmax=0.5,
    xticklabels=False,
    yticklabels=False,
    cbar_kws={"label": "Similaridade"},
)

ax.set_title(
    "Heatmap de Similaridade de Cosseno",
    fontsize=14,
    fontweight="bold"
)

fig.tight_layout()

salvar(fig, "05_heatmap")

# =====
# [6] HISTOGRAMA DOS SCORES
# =====

print("[6] Histograma")

mascara = ~np.eye(matriz_sim.shape[0], dtype=bool)

scores = matriz_sim[mascara]

scores = scores[scores > 0.01]

fig, ax = plt.subplots(figsize=(9, 4.5))

ax.hist(
    scores,
    bins=60,
    color=VERDE_CLARO,
    edgecolor=VERDE_ESCURO
)

media = scores.mean()

mediana = np.median(scores)

ax.axvline(
    media,

```

```

        color=DESTAQUE,
        linestyle="--",
        linewidth=2,
        label=f"Média = {media:.3f}"
    )

    ax.axvline(
        mediana,
        color=VERDE_ESCURO,
        linestyle=":",
        linewidth=2,
        label=f"Mediana = {mediana:.3f}"
    )

    ax.legend()

    ax.set_title(
        "Distribuição dos Scores de Similaridade",
        fontsize=14,
        fontweight="bold"
    )

    ax.set_xlabel("Similaridade")

    ax.set_ylabel("Frequência")

    fig.tight_layout()

    salvar(fig, "06_histograma_scores")

    # =====
    # [7] MELHOR SCORE POR ARTIGO
    # =====

    print("[7] Melhor score por artigo")

    top_scores = []

    for i in range(len(df)):

        linha = matriz_sim[i].copy()

        linha[i] = 0

        top_scores.append(linha.max())

    top_scores = np.array(top_scores)

    fig, ax = plt.subplots(figsize=(9, 4.5))

    ax.hist(
        top_scores,
        bins=40,
        color=VERDE_MEDIO,
        edgecolor=VERDE_ESCURO
    )

    ax.axvline(
        top_scores.mean(),
        color=DESTAQUE,
        linestyle="--",

```

```

        linewidth=2,
        label=f"Média = {top_scores.mean():.3f}"
    )

ax.legend()

ax.set_title(
    "Melhor Score de Similaridade por Artigo",
    fontsize=14,
    fontweight="bold"
)

ax.set_xlabel("Score")

ax.set_ylabel("Quantidade de Artigos")

fig.tight_layout()

salvar(fig, "07_melhor_score")

# =====
# [8] TOP 10 PARES MAIS SIMILARES
# =====

print("[8] Top pares")

pares = []

for i in range(len(df)):
    for j in range(i + 1, len(df)):
        pares.append((matriz_sim[i, j], i, j))

pares.sort(reverse=True)

top10 = pares[:10]

rotulos = []
valores = []

for score, i, j in top10:
    t1 = df.iloc[i]["titulo"][:35]
    t2 = df.iloc[j]["titulo"][:35]

    rotulos.append(f"{t1}\n↔ {t2}")

    valores.append(score)

fig, ax = plt.subplots(figsize=(12, 6))

bars = ax.barh(
    list(reversed(rotulos)),
    list(reversed(valores)),
    color=VERDE_CLARO,
    edgecolor=VERDE_ESCURO
)

ax.set_title(
    "Top 10 Pares Mais Similares",

```

```

        fontsize=14,
        fontweight="bold"
    )

    ax.set_xlabel("Similaridade")

    fig.tight_layout()

    salvar(fig, "08_top10_pares")

    # =====
    # [9] PCA 2D
    # =====

    print("[9] PCA")

    pca = PCA(n_components=2, random_state=42)

    coords = pca.fit_transform(tfidf_norm.toarray())

    variancia = pca.explained_variance_ratio_ * 100

    fig, ax = plt.subplots(figsize=(10, 7))

    scatter = ax.scatter(
        coords[:, 0],
        coords[:, 1],
        c=df["ano"],
        cmap="YlGn",
        s=40,
        alpha=0.7,
        edgecolors=VERDE_ESCURO,
        linewidths=0.3
    )

    cbar = fig.colorbar(scatter)

    cbar.set_label("Ano")

    ax.set_title(
        "Dispersão 2D do Corpus via PCA",
        fontsize=14,
        fontweight="bold"
    )

    ax.set_xlabel(
        f"PC1 ({variancia[0]:.1f}% variância)"
    )

    ax.set_ylabel(
        f"PC2 ({variancia[1]:.1f}% variância)"
    )

    fig.tight_layout()

    salvar(fig, "09_pca_2d")

    # =====
    # FINAL
    # =====

```

```
print("\n" + "=" * 65)
print("VISUALIZAÇÕES CONCLUÍDAS")
print("=" * 65)

print(f"✓ Gráficos salvos em: {PASTA_SAIDA}")

print("\nArquivos gerados:\n")

for arq in sorted(os.listdir(PASTA_SAIDA)):
    print(f"• {arq}")
```